COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/19/26	**DATE SUBMITTED:**	10/19/26
NAME:	Rayven Ferry Licas	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

5. Procedure

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Since all threads must be given a function to complete at their creation, we first must declare a function for it to be given. We'll name this function print and will design it to take int and string arguments when called. When executed, this code will simply report the data values passed in.

```
void print(int n, const std::string &str) {
std::cout << "Printing integer: " << n << std::endl;
std::cout << "Printing string: " << str << std::endl;
}
```

In the next section, we'll initialize a thread and have it execute the above function. To do this, we'll have the main function, the default executor present in all C++ applications, initialize the thread for the print function. After that, we use another handy multithreading command, join(), pausing the main function's thread until the specified thread, in this case t1, has finished its task. Without join() here, the main thread would finish its task before t1 would complete print, resulting in an error.

```
int main() {
std::thread t1(print, 10, "T.I.P.");
t1.join();
return 0;
}
```

Show your output and analysis in section 6 as its own table.

Part 2: Multi-Threaded Example

While the outcome of the single thread example above could easily be replicated without using multithreaded code, we can truly see concurrency's benefits when we attempt to complete print multiple times with different sets of data. Without multithreading, this would be done by simply having the main thread repeat print one at a time until completion.

To do this with concurrency in mind, we instead use a for loop to initialize multiple threads, pass them the print function and arguments, which they then complete concurrently. This multithreading option would be faster one using only the main thread as more of the total CPU is being used.

Runtime difference between multithreading and non-multithreading solutions increasing as more print executions are needed.

Let's see what a many-thread version of the above code would look like:

```
void print(int n, const std::string &str) {
std::string msg = std::to_string(n) + " : " + str;
std::cout << msg << std::endl;
}
int main() {
std::vector<std::string> s = {
"T.I.P.",
"Competent",
"Computer",
"Engineers"
};
std::vector<std::thread> threads;
for (int i = 0; i < s.size(); i++) {
threads.push_back(std::thread(print, i, s[i]));
}
for (auto &th : threads) {
th.join();
}
return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SCREENSHOT	`#include <iostream>`
	`#include <thread>`
	`#include <string>`
	
	`void print(int n, const std::string &str) {`
	`  std::cout << "Printing integer: " << n << std::endl;`
	`  std::cout << "Printing string: " << str << std::endl;`
	`}`
	
	`int main() {`
	`  std::thread t1(print, 10, "T.I.P.");`
	
	`  t1.join();`
	
	`  return 0;`
	`}`

ANALYSIS	In this program, the main execution thread creates a

	worker thread named t1 and assigns it to run the print() function using 10 and "T.I.P." as inputs. Calling t1.join() pauses the main program so it waits for t1 to finish its task completely. Without t1.join(), the main program would close before t1 finishes printing, which causes the program to crash with an error.

Table 13-1. Simple One-Threaded Example

SCREENSHOT	```
	#include <iostream>
	#include <thread>
	#include <vector>
	#include <string>
	
	void print(int n, const std::string &str) {
	    std::string msg = std::to_string(n) + ":" +
	str;
	    std::cout << msg << std::endl;
	}
	
	int main() {
	    std::vector<std::string> s = {
	        "T.I.P.",
	        "Competent",
	        "Computer",
	        "Engineers"
	    };
	
	    std::vector<std::thread> threads;
	
	    for (int i = 0; i < s.size(); i++) {
	        threads.push_back(std::thread(print, i,
	s[i]));
	    }
	
	    for (auto &th : threads) {
	        th.join();
	    }
	
	    return 0;
	}
	``

ANALYSIS	In this multi-threaded program, a for loop creates several worker threads at once and gives each thread a different item from the string list to print concurrently. Running these tasks simultaneously uses more CPU power and finishes faster than printing them one by one in a single thread. A second for loop uses th.join() on every thread to ensure all worker threads finish

	before the main program ends. Because the computer processes the threads at the same time, the printed outputs can sometimes appear in a slightly different order each time you run it.

Table 13-2. Multithreaded Example

Supplementary Activity
Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.
Questions:
1. Write a definition of multithreading and its advantages/disadvantages.
Multithreading is a programming technique that allows a program to do multiple tasks at the same time. It helps the CPU work better, makes programs respond faster, and uses less memory than running separate processes. However, it can make the code harder to understand and fix. It can also cause errors when multiple threads use the same data at the same time.
2. Rationalize the use of multithreading by providing at least 3 use-cases.
Multithreading enables a program to execute multiple sub-tasks concurrently across available CPU cores to boost system responsiveness and processing performance—making it ideal for responsive GUIs, multi-user web servers, and game engine processing—whereas parallelism physically executes independent tasks simultaneously across separate hardware cores. Although multithreading can introduce data races when sharing memory without proper locks, sequentially calling `join()` on each thread forces the main thread to wait for tasks to finish before reading a global variable, guaranteeing predictable step-by-step updates; applying this to an array sum algorithm splits data into two halves processed concurrently on separate threads, summing each portion independently to speed up overall execution without memory conflicts.
3. Differentiate between parallelism and concurrency
Concurrency and parallelism are two distinct ways computers handle multiple tasks. Concurrency works by overlapping tasks and switching between them on a shared memory system, making it look like they are running at the same time. In contrast, parallelism actually executes tasks at the exact same physical time by assigning each task to a separate CPU core with its own set of data.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.
Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
Display the value of global variable at different combinations of using the join() per thread. Such that:
o Display
o T1.join()
o Display
o T2.join()
o Display
o T3.join()
Provide an analysis based on the outputs of the multi-threading exercise.

```
#include <iostream>
#include <thread>
```

```
int global_counter = 0; // Global variable

void add(int value) {
    global_counter += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Initial Display: " << global_counter << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << global_counter << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << global_counter << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << global_counter << std::endl;

    return 0;
}
```

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

```
#include <iostream>
#include <vector>
#include <thread>
#include <numeric>

void partial_sum(const std::vector<int>& arr, int start, int end, int&
result) {
    result = 0;
    for (int i = start; i < end; ++i) {
        result += arr[i];
    }
}
```

```cpp
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8};
    int sum1 = 0, sum2 = 0;
    int mid = numbers.size() / 2;

    std::thread t1(partial_sum, std::cref(numbers), 0, mid,
std::ref(sum1));
    std::thread t2(partial_sum, std::cref(numbers), mid, numbers.size(),
std::ref(sum2));

    t1.join();
    t2.join();

    int total = sum1 + sum2;
    std::cout << "First Half Sum: " << sum1 << std::endl;
    std::cout << "Second Half Sum: " << sum2 << std::endl;
    std::cout << "Total Parallel Sum: " << total << std::endl;

    return 0;
}
```

This program splits an 8-number list into two equal parts and calculates each part at the same time using two separate threads. Thread `t1` adds the first half to get `10`, while thread `t2` adds the second half to get `26` without interfering with each other. Adding both results together gives the final total of `36` much faster by using different CPU cores at once.

8. Conclusion

In this activity, I learned how to use `std::thread` and `join()` in C++ to run multiple tasks at the same time across CPU cores. I analyzed how running threads concurrently speeds up programs, while using `join()` prevents errors and keeps execution under control. Through the supplementary tasks, I understood the key differences between concurrency and parallelism and saw how splitting array work across threads improves performance. Overall, I did very well on this activity, though I can improve by learning how to use mutexes to safely handle shared data between threads.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.